

On the Simplex Method

Agni Datta*

University of Edinburgh, UK

agnidatta.org@gmail.com

Abstract. We study the simplex method as the central pivoting method for linear programming. We start from the geometry of polyhedra and basic feasible solutions, derive the primal simplex method from reduced costs, and prove the optimality and pivoting criteria that justify the algorithm. We then develop the dual simplex method, the two-phase method, and the Big- M method, emphasizing what each variant preserves during its pivots and why each one is useful in computation. The last part discusses revised simplex implementations, sparse basis updates, network and bounded-variable variants, decomposition and column generation, and the complexity theory that explains why simplex can be both exponentially bad in the worst case and effective in practice [Dan51, Dan63, Chv83, BT97, Sch86, Mar03, ST04].

*Version 0.1. This manuscript is a working note. Corrections, suggestions, and references to related work are welcome.

Contents

Abstract	1
1 Introduction	4
2 Linear Programs and Polyhedral Geometry	4
2.1 Standard Form	4
2.2 Existence of an Optimal Basis	5
3 Reduced Costs and Optimality	6
3.1 Dictionary Associated with a Basis	6
3.2 The Dual Certificate	7
4 The Primal Simplex Method	7
4.1 Entering and Leaving Variables	7
4.2 The Algorithm and Degeneracy	8
5 Dual Simplex	9
5.1 Dual Feasible Dictionaries	9
5.2 Reoptimization and Certificates	10
6 Finding an Initial Basis	11
6.1 The Two-Phase Method	11
6.2 The Big- M Method	12
7 Revised Simplex and Numerical Implementation	13
7.1 What the Revised Method Computes	13
7.2 Basis Updates and Numerical Stability	13
7.3 Pricing, Steepest Edge, and Practical Pivoting	14
8 Variants of the Simplex Method	14
8.1 Bounded-Variable Simplex	14
8.2 Network Simplex	15
8.3 Dual Simplex in Branch-and-Bound	15
8.4 Column Generation and Decomposition	15
8.5 Criss-Cross and Other Basis-Exchange Methods	16
9 Complexity and Smoothed Behavior	16
9.1 Worst-Case Exponential Behavior	16
9.2 Polynomial-Time Linear Programming	17
9.3 Smoothed Analysis	17
10 What the Simplex Method Teaches	17
Acknowledgements	19

1 Introduction

Linear programming became useful because it gave a language in which economic allocation, production planning, transportation, and combinatorial optimization could be written as questions about polyhedra [Dan51, Dan63, Chv83, Sch86]. The simplex method was Dantzig’s algorithmic answer to this language. It does not search the entire feasible region. It moves from one vertex to an adjacent vertex, using the objective function to decide which edge to follow, and it stops when the local certificate of improvement disappears.

This description omits the computational issue. A polyhedron may have exponentially many vertices, and so an algorithm that moves among vertices should not automatically be expected to be efficient. Nevertheless, simplex remains central in numerical optimization, both historically and practically [Dan63, Mar03, Van20]. Its effectiveness comes from structural facts, implementation ideas, and empirical regularities rather than from a single worst-case theorem. Bases can be updated cheaply; sparse linear algebra can be exploited; degeneracy can be controlled; dual feasibility can be preserved after adding constraints; network and bounded-variable structure can be used directly; and column generation can let the algorithm work with a small active part of an enormous model [BG69, FT72, AMO93, DW60, GG61].

The history also contains a warning. Klee and Minty showed that natural pivot rules may take exponentially many steps [KM72], while Khachiyan’s ellipsoid method and Karmarkar’s interior-point method showed that linear programming has polynomial-time algorithms by routes very different from simplex [Kha79, Kar84]. Smoothed analysis later explained why worst-case examples do not by themselves describe ordinary numerical behavior [ST04], and probabilistic analyses had already pointed in a similar direction [Bor87]. We shall therefore treat simplex as both a mathematical algorithm and a computational framework. The mathematics explains its certificates and pivots; the framework explains its many variants.

2 Linear Programs and Polyhedral Geometry

The simplex method is easiest to understand after one separates two roles played by a linear program. The inequalities define a polyhedron, while the objective function orients the polyhedron by declaring which edges improve the objective. The algorithm works because, under mild hypotheses, an optimum of a linear function over a polyhedron is attained at a face, and when the optimum is finite and the feasible region has a vertex, some optimal vertex exists.

2.1 Standard Form

We work primarily with the standard-form linear program

$$\text{LP} \quad \min \mathbf{c}^\top \mathbf{x} \quad \text{subject to} \quad \mathbf{Ax} = \mathbf{b}, \quad \mathbf{x} \geq 0, \tag{2.1}$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ has rank m , the vector $\mathbf{b} \in \mathbb{R}^m$ is fixed, and the vector $\mathbf{c} \in \mathbb{R}^n$ gives the objective. Inequality systems can be transformed to Equation (2.1) by adding slack variables, splitting free variables into differences of nonnegative variables, and replacing maximization by minimization. These transformations change the representation, not the geometric problem.

The feasible region of Equation (2.1) is

$$\mathcal{P} = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{b}, \mathbf{x} \geq 0\}. \quad (2.2)$$

The affine equations restrict $\mathbf{x}$ to an affine subspace, while the non-negativity constraints cut that subspace by coordinate halfspaces. Thus, every face of $\mathcal{P}$ is obtained by setting some coordinates to zero.

Definition 2.1 (Basis and basic solution). A basis is a set $\mathcal{B} \subseteq [n]$ of size m such that the submatrix $\mathbf{B} = \mathbf{A}_{\mathcal{B}}$ is nonsingular. The complement $\mathcal{N} = [n] \setminus \mathcal{B}$ is the non-basis. The basic solution associated with $\mathcal{B}$ is obtained by setting $\mathbf{x}_{\mathcal{N}} = 0$ and solving

$$\mathbf{x}_{\mathcal{B}} = \mathbf{B}^{-1}\mathbf{b}. \quad (2.3)$$

If $\mathbf{x}_{\mathcal{B}} \geq 0$, then the basic solution is a basic feasible solution, abbreviated BFS. ┘

A basis is algebraic data, whereas a vertex is geometric data. The following result says that, in standard form, these are the same object up to degeneracy.

Proposition 2.2 (Vertices and basic feasible solutions). A point $\mathbf{x} \in \mathcal{P}$ is a vertex of $\mathcal{P}$ if and only if $\mathbf{x}$ is a BFS for some basis. ┘

Proof. The proof uses the fact that a point is not a vertex exactly when it can move in a nonzero feasible direction in both signs. The active non-negativity constraints at $\mathbf{x}$ determine which coordinates are forced to remain zero under such a motion, and the equality constraints determine whether the remaining coordinates have any freedom. A basis is precisely a choice of m columns that kills this freedom.

Suppose first that $\mathbf{x}$ is the BFS associated with a basis $\mathcal{B}$. Then $\mathbf{x}_{\mathcal{N}} = 0$ and $\mathbf{B}\mathbf{x}_{\mathcal{B}} = \mathbf{b}$. If $\mathbf{x} = (\mathbf{y} + \mathbf{z})/2$ for two feasible points $\mathbf{y}, \mathbf{z} \in \mathcal{P}$, then for every $j \in \mathcal{N}$ we have $0 = x_j = (y_j + z_j)/2$. Since $\mathbf{y}, \mathbf{z} \geq 0$, this implies $y_j = z_j = 0$ for all $j \in \mathcal{N}$. Therefore, both $\mathbf{y}$ and $\mathbf{z}$ have the same non-basic coordinates as $\mathbf{x}$. The equality constraints give $\mathbf{B}\mathbf{y}_{\mathcal{B}} = \mathbf{b}$ and $\mathbf{B}\mathbf{z}_{\mathcal{B}} = \mathbf{b}$, and nonsingularity of $\mathbf{B}$ gives $\mathbf{y}_{\mathcal{B}} = \mathbf{z}_{\mathcal{B}} = \mathbf{B}^{-1}\mathbf{b} = \mathbf{x}_{\mathcal{B}}$. Hence, $\mathbf{y} = \mathbf{z} = \mathbf{x}$, and $\mathbf{x}$ is a vertex.

Conversely, suppose $\mathbf{x} \in \mathcal{P}$ is not representable as a BFS. Let $S = \{j : x_j > 0\}$ be the support of $\mathbf{x}$. If the columns $\{\mathbf{A}_j : j \in S\}$ are linearly independent, then extend S to a basis $\mathcal{B}$ using columns of $\mathbf{A}$, and observe that $\mathbf{x}_{[n] \setminus \mathcal{B}} = 0$, so $\mathbf{x}$ is the BFS associated with $\mathcal{B}$. Thus, since $\mathbf{x}$ is not a BFS, the columns indexed by S are linearly dependent. There exists a nonzero vector $\mathbf{d}$ supported on S such that $\mathbf{A}\mathbf{d} = 0$. Since $x_j > 0$ for all $j \in S$, there is $\epsilon > 0$ such that $\mathbf{x} + \epsilon\mathbf{d} \geq 0$ and $\mathbf{x} - \epsilon\mathbf{d} \geq 0$. Both points satisfy the equality constraints because $\mathbf{A}\mathbf{d} = 0$, and they are distinct feasible points whose average is $\mathbf{x}$. Therefore, $\mathbf{x}$ is not a vertex. □

2.2 Existence of an Optimal Basis

The simplex method would be unjustified if an optimum could hide in the interior of a high-dimensional face without any optimal vertex. The next proposition rules this out in the setting used by the algorithm.

Proposition 2.3 (Fundamental theorem of linear programming). If Equation (2.1) has a feasible solution and a finite optimum, and if $\mathcal{P}$ has at least one vertex, then some BFS is optimal. ┘

Proof. The proof follows the objective down to a minimal face. A linear objective is constant on any

face on which it attains its minimum, and every nonempty polyhedron with a vertex contains a vertex in each nonempty face that is minimal under inclusion.

Let α be the optimal value, and let

$$F = \{\mathbf{x} \in \mathcal{P} : \mathbf{c}^\top \mathbf{x} = \alpha\}$$

be the optimal face. The set F is nonempty by the assumption that the optimum is attained; in the usual algorithmic setting this follows from feasibility, boundedness in the objective direction, and closedness of $\mathcal{P}$. Choose a face G of $\mathcal{P}$ that is contained in F and minimal among nonempty faces contained in F . Such a face exists because a polyhedron has only finitely many faces. The minimal face G has dimension zero; otherwise, if it contained two distinct points, the line segment between them would expose a proper nonempty face of G by activating an additional valid inequality. Thus, $G = \{\mathbf{x}^*\}$ is a vertex. By [Proposition 2.2](#), $\mathbf{x}^*$ is a BFS, and by construction $\mathbf{c}^\top \mathbf{x}^* = \alpha$. $\square$

[Proposition 2.3](#) is the geometric reason simplex can restrict attention to bases. The algorithm may visit many bases, but it does not lose optimality by ignoring non-vertex feasible points.

3 Reduced Costs and Optimality

The simplex method needs a local certificate that a basis is optimal and, when it is not optimal, a local direction that improves the objective. Both are encoded by reduced costs. This is the first point at which duality appears, although one can derive the calculation without explicitly introducing the dual program.

3.1 Dictionary Associated with a Basis

Fix a basis $\mathcal{B}$ with matrix $\mathbf{B} = \mathbf{A}_{\mathcal{B}}$ and nonbasis $\mathcal{N}$. Partition the variables, objective vector, and constraint matrix as

$$\mathbf{x} = (\mathbf{x}_{\mathcal{B}}, \mathbf{x}_{\mathcal{N}}), \quad \mathbf{c} = (\mathbf{c}_{\mathcal{B}}, \mathbf{c}_{\mathcal{N}}), \quad \mathbf{A} = (\mathbf{B}, \mathbf{N}).$$

The equality constraints become

$$\mathbf{B}\mathbf{x}_{\mathcal{B}} + \mathbf{N}\mathbf{x}_{\mathcal{N}} = \mathbf{b},$$

and hence

$$\mathbf{x}_{\mathcal{B}} = \mathbf{B}^{-1}\mathbf{b} - \mathbf{B}^{-1}\mathbf{N}\mathbf{x}_{\mathcal{N}}. \tag{3.1}$$

Substituting [Equation \(3.1\)](#) into the objective gives

$$\mathbf{c}^\top \mathbf{x} = \mathbf{c}_{\mathcal{B}}^\top \mathbf{B}^{-1}\mathbf{b} + (\mathbf{c}_{\mathcal{N}}^\top - \mathbf{c}_{\mathcal{B}}^\top \mathbf{B}^{-1}\mathbf{N}) \mathbf{x}_{\mathcal{N}}. \tag{3.2}$$

The row vector

$$\bar{\mathbf{c}}_{\mathcal{N}}^\top = \mathbf{c}_{\mathcal{N}}^\top - \mathbf{c}_{\mathcal{B}}^\top \mathbf{B}^{-1}\mathbf{N} \tag{3.3}$$

is the reduced-cost vector of the non-basic variables.

Proposition 3.1 (Reduced-cost optimality criterion). Let $\mathcal{B}$ be a feasible basis for the minimization problem in [Equation \(2.1\)](#). If every reduced cost satisfies $\bar{c}_j \geq 0$ for $j \in \mathcal{N}$, then the associated BFS is optimal. $\square$

Proof. The proof is a one-line inequality, but it is the central inequality of simplex. Once the basic variables have been eliminated, every feasible point is obtained by choosing nonnegative non-basic variables, and nonnegative reduced costs ensure that every such choice can only increase the objective.

Let $\mathbf{x} \in \mathcal{P}$ be feasible. Since $\mathbf{x}_{\mathcal{N}} \geq 0$ and $\bar{\mathbf{c}}_{\mathcal{N}} \geq 0$ componentwise, Equation (3.2) gives

$$\mathbf{c}^\top \mathbf{x} = \mathbf{c}_{\mathcal{B}}^\top \mathbf{B}^{-1} \mathbf{b} + \bar{\mathbf{c}}_{\mathcal{N}}^\top \mathbf{x}_{\mathcal{N}} \geq \mathbf{c}_{\mathcal{B}}^\top \mathbf{B}^{-1} \mathbf{b}.$$

The basic feasible solution $\mathbf{x}^{\mathcal{B}}$ has $\mathbf{x}_{\mathcal{N}}^{\mathcal{B}} = 0$ and $\mathbf{x}_{\mathcal{B}}^{\mathcal{B}} = \mathbf{B}^{-1} \mathbf{b}$, so its objective value is exactly $\mathbf{c}_{\mathcal{B}}^\top \mathbf{B}^{-1} \mathbf{b}$. Hence, no feasible point has smaller objective value. $\square$

3.2 The Dual Certificate

The reduced-cost condition is also the dual-feasibility condition in disguise. The dual of Equation (2.1) is

$$\max \mathbf{b}^\top \mathbf{y} \quad \text{subject to} \quad \mathbf{A}^\top \mathbf{y} \leq \mathbf{c}. \quad (3.4)$$

Given a basis $\mathcal{B}$, define the multiplier vector

$$\mathbf{y}_{\mathcal{B}} = \mathbf{B}^{-\top} \mathbf{c}_{\mathcal{B}}. \quad (3.5)$$

Then $\mathbf{B}^\top \mathbf{y}_{\mathcal{B}} = \mathbf{c}_{\mathcal{B}}$, and for every non-basic column j we have

$$c_j - \mathbf{A}_j^\top \mathbf{y}_{\mathcal{B}} = \bar{c}_j.$$

Thus, nonnegative reduced costs say exactly that $\mathbf{y}_{\mathcal{B}}$ is feasible for the dual. Since

$$\mathbf{b}^\top \mathbf{y}_{\mathcal{B}} = \mathbf{c}_{\mathcal{B}}^\top \mathbf{B}^{-1} \mathbf{b},$$

primal and dual objective values agree at the current basis, and optimality follows by weak duality. This dual reading is the reason the same algebra supports both primal and dual simplex.

4 The Primal Simplex Method

The primal simplex method maintains primal feasibility and tries to repair dual infeasibility. In a minimization problem, a negative reduced cost says that increasing the corresponding non-basic variable decreases the objective. The pivot chooses such an entering variable and then increases it until some current basic variable reaches zero, which preserves feasibility while moving to an adjacent basis.

4.1 Entering and Leaving Variables

Assume $\mathcal{B}$ is feasible and let $q \in \mathcal{N}$ satisfy $\bar{c}_q < 0$. We consider increasing x_q from zero while keeping all other non-basic variables zero. By Equation (3.1), the basic variables change as

$$\mathbf{x}_{\mathcal{B}}(t) = \mathbf{B}^{-1} \mathbf{b} - t \mathbf{B}^{-1} \mathbf{A}_q, \quad x_q(t) = t, \quad x_j(t) = 0 \quad \text{for } j \in \mathcal{N} \setminus \{q\}. \quad (4.1)$$

Let $\mathbf{d} = \mathbf{B}^{-1}\mathbf{A}_q$. Feasibility requires

$$\mathbf{B}^{-1}\mathbf{b} - t\mathbf{d} \geq 0, \quad t \geq 0.$$

If $d_i \leq 0$ for every i , then feasibility holds for all $t \geq 0$, and the objective decreases without bound because $\bar{c}_q < 0$. Otherwise, the largest feasible step is

$$\theta = \min \left\{ \frac{(\mathbf{B}^{-1}\mathbf{b})_i}{d_i} : d_i > 0 \right\}. \quad (4.2)$$

Any index attaining the minimum in Equation (4.2) may leave the basis.

Proposition 4.1 (Primal pivot step). Suppose $\mathcal{B}$ is a feasible basis, $\bar{c}_q < 0$ for some $q \in \mathcal{N}$, and $\mathbf{d} = \mathbf{B}^{-1}\mathbf{A}_q$ has at least one positive component. Replacing a leaving index $p \in \mathcal{B}$ that attains Equation (4.2) by q produces another feasible basis. If the step length θ is positive, then the objective value decreases by $-\theta\bar{c}_q > 0$. $\square$

Proof. The ratio test chooses the largest movement along the improving edge that keeps all basic variables nonnegative. The leaving variable is precisely a coordinate that becomes tight at that largest movement, and exchanging it for the entering variable gives the adjacent basis.

For $0 \leq t \leq \theta$, the definition of θ gives $(\mathbf{B}^{-1}\mathbf{b})_i - td_i \geq 0$ whenever $d_i > 0$, while the same inequality holds automatically when $d_i \leq 0$. Thus, the point in Equation (4.1) is feasible for every $t \in [0, \theta]$. At $t = \theta$, the leaving coordinate p satisfies $(\mathbf{B}^{-1}\mathbf{b})_p - \theta d_p = 0$. Since $d_p > 0$, the column $\mathbf{A}_q$ has a nonzero coefficient in the expression $\mathbf{B}^{-1}\mathbf{A}_q$, and the usual column-exchange argument shows that replacing column p of $\mathbf{B}$ by $\mathbf{A}_q$ yields a nonsingular basis matrix.

Using Equation (3.2) along the edge, the objective value is

$$\mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{b} + \bar{c}_q t.$$

At $t = \theta$ this equals the old objective value plus $\bar{c}_q\theta$. Since $\bar{c}_q < 0$ and $\theta > 0$, the objective strictly decreases by $-\bar{c}_q\theta$. $\square$

4.2 The Algorithm and Degeneracy

The primal simplex method repeats the reduced-cost and ratio-test calculation until either all reduced costs are nonnegative, in which case Proposition 3.1 proves optimality, or an improving direction has no positive component in $\mathbf{B}^{-1}\mathbf{A}_q$, in which case the primal problem is unbounded. This gives a complete finite-state algorithm once a pivot rule is specified.

Degeneracy is the obstruction to a naive termination proof. A basis is degenerate when at least one basic variable has value zero. Then the ratio test may return $\theta = 0$, and the algorithm may change bases without changing the point or the objective value. Cycling examples show that this is a real issue [Bea54, Chv83]. Bland's rule, which chooses the smallest indexed entering variable among eligible variables and then the smallest indexed leaving variable among ties, gives a finite pivoting rule [Bla77]. Lexicographic perturbation gives another standard anti-cycling mechanism, and it is often the cleaner mathematical model for implementations [Chv83, Mar03].

Theorem 4.2 (Finite termination under Bland's rule [Bla77]). For a feasible bounded linear program in standard form, the primal simplex method with Bland's pivot rule terminates at an optimal basis after finitely many pivots. $\square$

Proof sketch. The proof assigns an order-theoretic certificate to each basis and shows that no basis can repeat under Bland's rule. Since there are only finitely many bases, this excludes cycling, and every non-terminal pivot either preserves feasibility and changes the basis or certifies unboundedness.

We recall the mechanism. Suppose, toward contradiction, that a cycle of bases occurs. Along the cycle, the objective value must remain constant, because any positive pivot step with a negative reduced cost would strictly decrease the objective and prevent return. Hence, every pivot in the cycle is degenerate. Bland's smallest-index choices imply that, among all variables that enter or leave during the cycle, the largest indexed one cannot leave and later re-enter without contradicting the reduced-cost sign pattern at the pivot where it re-enters. This contradiction rules out cycles. The full argument is a careful bookkeeping proof over the indices of variables and reduced costs; see Bland's original paper or the standard treatments in Chvátal and Vanderbei [Bla77, Chv83, Van20]. $\square$

The theorem is not a complexity theorem. It proves that the algorithm stops, not that it stops quickly. The distinction matters because the number of bases is finite but may be exponential in the input size.

5 Dual Simplex

The dual simplex method reverses the invariant maintained by primal simplex. Primal simplex keeps $\mathbf{B}^{-1}\mathbf{b} \geq 0$ and repairs negative reduced costs. Dual simplex keeps the reduced costs nonnegative and repairs primal infeasibility. This reversal is especially useful after adding a new constraint, tightening a bound, or branching in an integer-programming solver, because such operations often preserve dual feasibility while destroying primal feasibility [Mar03, Van20].

It is useful to regard the dual method as a simplex method applied to the dual program while still computing in the primal dictionary. The current basis determines the dual multiplier $\mathbf{y} = \mathbf{B}^{-\top}\mathbf{c}_{\mathcal{B}}$, and nonnegative reduced costs say that this multiplier is dual feasible. A negative basic value does not invalidate the dual certificate; it says instead that complementary slackness cannot yet be realized by a primal feasible point. Dual simplex therefore moves along adjacent dual feasible bases until the primal basic vector becomes nonnegative.

5.1 Dual Feasible Dictionaries

Let $\mathcal{B}$ be a basis for which

$$\bar{\mathbf{c}}_{\mathcal{N}} \geq 0 \tag{5.1}$$

but $\mathbf{B}^{-1}\mathbf{b}$ may have negative components. If $\mathbf{B}^{-1}\mathbf{b} \geq 0$ also holds, then the basis is primal feasible and optimal by Proposition 3.1. Otherwise, choose a row p with $(\mathbf{B}^{-1}\mathbf{b})_p < 0$. The leaving variable is the infeasible basic variable indexed by p .

To restore feasibility while preserving nonnegative reduced costs, dual simplex chooses an entering variable from the non-basic variables whose coefficient in row p permits the infeasible basic variable to increase. Writing the dictionary row as

$$x_{B_p} = \bar{b}_p - \sum_{j \in \mathcal{N}} \bar{a}_{pj} x_j, \tag{5.2}$$

we need $\bar{a}_{pj} < 0$ in order for increasing x_j to raise x_{B_p} . The dual ratio test chooses an entering index

q minimizing

$$\frac{\bar{c}_j}{-\bar{a}_{pj}} \quad \text{over all } j \in \mathcal{N} \text{ with } \bar{a}_{pj} < 0. \quad (5.3)$$

The ratio in Equation (5.3) has the same meaning as the primal ratio test, but it is measured in the dual objective geometry. The row p is fixed by primal infeasibility, and the admissible columns are exactly the columns that can remove that infeasibility. Among them, the minimum ratio is the largest dual step that does not make any reduced cost negative.

Proposition 5.1 (Dual pivot step). If the current dictionary is dual feasible and has a row p with $\bar{b}_p < 0$, then a pivot chosen by Equation (5.3) preserves dual feasibility. If no index j satisfies $\bar{a}_{pj} < 0$, then the primal problem is infeasible. $\square$

Proof. The dual ratio test is the same reduced-cost calculation seen from the transposed side. A pivot changes the dual multiplier by moving along a dual edge; choosing the smallest admissible ratio is exactly what keeps all new reduced costs nonnegative.

If no $\bar{a}_{pj} < 0$ exists, then the row equation gives

$$x_{B_p} = \bar{b}_p - \sum_{j \in \mathcal{N}} \bar{a}_{pj} x_j < 0$$

for every $\mathbf{x}_{\mathcal{N}} \geq 0$, because $\bar{b}_p < 0$ and every term $-\bar{a}_{pj} x_j$ is nonpositive. Thus, no primal feasible solution exists.

Assume now that at least one admissible entering index exists. Pivoting on $\bar{a}_{pq} < 0$ expresses x_q as a basic variable and x_{B_p} as a non-basic variable. The reduced costs transform by subtracting a multiple of the pivot row. For every non-basic index j with $\bar{a}_{pj} < 0$, the choice of q gives

$$\frac{\bar{c}_q}{-\bar{a}_{pq}} \leq \frac{\bar{c}_j}{-\bar{a}_{pj}},$$

which rearranges to

$$\bar{c}_j - \bar{c}_q \frac{\bar{a}_{pj}}{\bar{a}_{pq}} \geq 0.$$

For indices with $\bar{a}_{pj} \geq 0$, the transformed reduced cost is nonnegative because $\bar{c}_j \geq 0$, $\bar{c}_q \geq 0$, and $\bar{a}_{pq} < 0$. The new reduced cost of the leaving variable is $\bar{c}_q / (-\bar{a}_{pq}) \geq 0$. Hence, all reduced costs remain nonnegative. $\square$

5.2 Reoptimization and Certificates

The reoptimization role of dual simplex is best seen through certificates. Suppose a basis is optimal for an old problem. If we add a new valid row, tighten a right-hand side, or branch on a variable bound, the old reduced costs often remain nonnegative because the objective and most columns have not changed. The basis is therefore still close to a dual certificate. What may fail is the equation $\mathbf{B}^{-1}\mathbf{b} \geq 0$, since the right-hand side or the active row system has changed. Dual simplex exploits exactly this situation by repairing primal feasibility while preserving the dual certificate [Mar03, BJS04, Van20].

This explains why dual simplex dominates many mixed-integer and cutting-plane workflows. A branch-and-bound tree produces a sequence of closely related LP relaxations, not a sequence of unrelated problems. A cutting-plane method adds rows to a relaxation and then resolves. In

both cases, the previous optimal basis is too valuable to discard. Dual simplex is the basis-repair mechanism that converts this inherited information into a new optimality certificate.

Dual simplex is therefore not a separate theory; it is the same basis algebra with a different invariant. Modern solvers switch between primal and dual simplex depending on which invariant is cheaper to restore after model changes [Mar03, Van20].

6 Finding an Initial Basis

The primal simplex method assumes a feasible basis, and the dual simplex method assumes a dual feasible basis. Real models seldom arrive with either certificate already visible. Phase methods supply such a certificate by solving an auxiliary problem whose purpose is not the original objective but feasibility.

This distinction is important in implementations. Finding an initial basis is not a preliminary annoyance; it is the first certificate problem. The auxiliary problem must distinguish infeasibility from poor initialization, and it must return a basis that can be transferred to the original objective without corrupting the dictionary. The two-phase method does this lexically, while the Big- M method does it by embedding the auxiliary objective into the original one [Chv83, BT97, Mar03].

6.1 The Two-Phase Method

Suppose the constraints are in equality form $\mathbf{Ax} = \mathbf{b}$ with $\mathbf{x} \geq 0$ and $\mathbf{b} \geq 0$. If no identity submatrix is available, introduce artificial variables $\mathbf{w} \geq 0$ and solve

$$\text{Phase I} \quad \min \mathbf{1}^\top \mathbf{w} \quad \text{subject to} \quad \mathbf{Ax} + \mathbf{w} = \mathbf{b}, \quad \mathbf{x} \geq 0, \quad \mathbf{w} \geq 0. \quad (6.1)$$

The artificial variables form an immediate feasible basis. If the optimum of Equation (6.1) is positive, then the original system has no feasible solution. If the optimum is zero, then all artificial variables can be removed or driven out of the basis by standard cleanup pivots, and the remaining basis initializes Phase II, which solves the original objective.

Proposition 6.1 (Correctness of Phase I). The original system $\mathbf{Ax} = \mathbf{b}$, $\mathbf{x} \geq 0$ is feasible if and only if the auxiliary problem in Equation (6.1) has optimal value zero. $\square$

Proof. The artificial variables measure the violation of feasibility in the equality system after all right-hand sides have been made nonnegative. Driving their sum to zero is exactly the condition that the original variables alone satisfy the constraints.

If the original system has a feasible solution $\mathbf{x}$, then $(\mathbf{x}, 0)$ is feasible for Equation (6.1) with objective value zero. Since $\mathbf{w} \geq 0$, every feasible point of Equation (6.1) has $\mathbf{1}^\top \mathbf{w} \geq 0$, so the optimum is zero.

Conversely, if Equation (6.1) has optimal value zero, then any optimal feasible pair $(\mathbf{x}, \mathbf{w})$ satisfies $\mathbf{1}^\top \mathbf{w} = 0$. Since each component of $\mathbf{w}$ is nonnegative, every component is zero. The constraint $\mathbf{Ax} + \mathbf{w} = \mathbf{b}$ then reduces to $\mathbf{Ax} = \mathbf{b}$ with $\mathbf{x} \geq 0$, so $\mathbf{x}$ is feasible for the original system. $\square$

The conclusion of Proposition 6.1 gives feasibility, but the algorithm still needs a clean basis for the original variables. If an artificial variable is non-basic at the end of Phase I, it can be discarded. If an artificial variable is basic with value zero, then its row is degenerate. One pivots it out if some original non-basic column has a nonzero coefficient in that row. If no such column exists, the

row is redundant for the original equality system and may be removed after checking that it is implied by the remaining rows. This cleanup is part of the mathematical correctness of Phase II, because the original simplex method expects a basis made of original columns rather than artificial columns [Chv83, Mar03, Van20].

Proposition 6.2 (Transfer from Phase I to Phase II). Suppose the auxiliary problem in Equation (6.1) has optimal value zero. After removing redundant zero rows and pivoting out zero-valued artificial basic variables whenever possible, the remaining dictionary contains a feasible basis for the original standard-form problem. $\square$

Proof. The proof formalizes the cleanup step. At a zero optimum of Phase I, every artificial variable has value zero. Thus, the current basic solution satisfies the original equalities once artificial columns are ignored. The only obstruction is that a zero artificial variable may still occupy a basic column.

Consider a row whose basic variable is an artificial variable w_i with value zero. If some original non-basic variable x_j has a nonzero coefficient in this row, pivoting on that coefficient replaces w_i by x_j . The pivot is degenerate because the right-hand side of the row is zero, so feasibility is preserved. If no original non-basic variable has a nonzero coefficient in the row, then the row contains only the artificial basic variable and artificial non-basic variables. Since all artificial variables are zero at the Phase I optimum, this row imposes no additional restriction on the original variables beyond the restrictions already represented by the other rows. Removing the row and the artificial basic column therefore preserves the set of original feasible solutions. Repeating this finite procedure eliminates all artificial basic variables or redundant rows, and the remaining basic columns form a feasible basis for the original problem. $\square$

6.2 The Big- M Method

The Big- M method folds the auxiliary objective into the original objective by penalizing artificial variables with a large coefficient. For a minimization problem, one solves

$$\min \mathbf{c}^\top \mathbf{x} + M \mathbf{1}^\top \mathbf{w} \quad \text{subject to} \quad \mathbf{A}\mathbf{x} + \mathbf{w} = \mathbf{b}, \quad \mathbf{x} \geq 0, \quad \mathbf{w} \geq 0, \quad (6.2)$$

where M is chosen larger than any objective variation that can be relevant among feasible original solutions. Conceptually, Big- M and Phase I-Phase II encode the same feasibility logic. Practically, Phase I-Phase II is cleaner because it avoids numerical instability caused by very large penalty coefficients [Chv83, Mar03, Van20].

The Big- M method is still worth studying because it clarifies how artificial variables interact with reduced costs. The artificial basis is feasible immediately, and the penalty forces artificial variables to leave the basis whenever the original system permits it. If an artificial variable remains positive at optimum, the original model is infeasible. If all artificial variables vanish, the remaining basis is a legitimate starting point for the original problem.

The phrase “large M ” should be read symbolically rather than numerically. If M is represented by an actual floating-point number, then the objective coefficients mix the scale of the original optimization problem with an artificial scale chosen only to enforce feasibility. This can distort reduced costs and aggravate roundoff error. A symbolic Big- M implementation treats objective values lexicographically, first minimizing $\mathbf{1}^\top \mathbf{w}$ and only then minimizing $\mathbf{c}^\top \mathbf{x}$, which is exactly the two-phase method in another notation [Chv83, BT97, Mar03]. This is why modern implementations prefer an explicit Phase I solve.

7 Revised Simplex and Numerical Implementation

The tableau form is excellent for proving the method, but it is not how large-scale simplex is implemented. The revised simplex method stores and updates information about $\mathbf{B}^{-1}$ implicitly, computes only the reduced costs and columns needed for the next pivot, and uses sparse matrix factorizations to control both memory and numerical error [BG69, FT72, Mar03].

The distinction between tableau simplex and revised simplex is not cosmetic. A full tableau contains $\mathbf{B}^{-1}\mathbf{N}$, which has $m(n-m)$ entries and is typically much denser than the original constraint matrix. A revised implementation keeps the basis matrix and its factorization, solves the linear systems that are actually needed, and delays or avoids computing columns that will never be priced. In large sparse models this distinction is the difference between a proof device and a usable algorithm [BG69, FT72, Mar03, Van20].

7.1 What the Revised Method Computes

At each iteration the revised method needs three quantities:

$$\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}, \quad \mathbf{y} = \mathbf{B}^{-\top}\mathbf{c}_B, \quad \bar{c}_j = c_j - \mathbf{A}_j^\top \mathbf{y}. \quad (7.1)$$

After choosing an entering column q , it also needs

$$\mathbf{d} = \mathbf{B}^{-1}\mathbf{A}_q \quad (7.2)$$

for the ratio test. None of these computations requires forming the full tableau. They require solving linear systems with coefficient matrix $\mathbf{B}$ or $\mathbf{B}^\top$.

Modern implementations maintain a sparse factorization of $\mathbf{B}$, update it after pivots, and occasionally refactor from scratch to control fill-in and numerical drift. In the simplest description, one stores a factorization

$$\mathbf{PBQ} = \mathbf{LU}, \quad (7.3)$$

where $\mathbf{P}$ and $\mathbf{Q}$ are permutations, $\mathbf{L}$ is triangular with controlled sparsity, and $\mathbf{U}$ is upper triangular. Solving with $\mathbf{B}$ or $\mathbf{B}^\top$ then reduces to triangular solves and permutations. The product-form inverse, Bartels–Golub updates, and Forrest–Tomlin updates are classical milestones in this direction [BG69, FT72, Mar03]. The important conceptual point is that simplex is a sequence of adjacent bases, while revised simplex is a sequence of sparse linear-algebra problems associated with those bases.

7.2 Basis Updates and Numerical Stability

A pivot replaces one column of $\mathbf{B}$ by another column. Recomputing Equation (7.3) from scratch after every pivot is reliable but expensive. Updating the factorization is cheaper, but repeated updates introduce fill-in and numerical drift. Practical codes therefore use a hybrid discipline. They update the factorization for many pivots, monitor stability and sparsity, and periodically refactor the current basis from the original columns [BG69, FT72, Mar03, Van20].

Finite precision also changes the meaning of feasibility and reduced costs. A computed basic value near zero may be treated as zero; a reduced cost near zero may be treated as dual feasible; a pivot element that is algebraically nonzero may be numerically unsafe. This is why production simplex codes use feasibility tolerances, Markowitz-type sparsity criteria, perturbations, and refactorization

tests. These devices do not alter the mathematical proof of simplex, but they determine whether the proof survives contact with floating-point arithmetic [Mar03, BJS04, Van20].

7.3 Pricing, Steepest Edge, and Practical Pivoting

The pivot rule has two jobs. It must preserve correctness, and it should reduce total running time. Dantzig’s rule chooses an entering variable with the most negative reduced cost in a minimization problem. Steepest-edge and Devex rules modify this choice by normalizing objective improvement by a measure of the edge length, which often reduces the number of pivots at the cost of more work per pivot [Mar03, Van20]. Partial pricing and multiple pricing reduce the cost of scanning all non-basic variables when n is large.

Degeneracy also shapes practical pivoting. Anti-cycling rules such as Bland’s rule are mathematically clean, but they may be too conservative for computation. Implementations usually combine perturbation, tolerance management, and heuristic pricing. These choices do not change the reduced-cost theory; they determine how robustly and quickly the theory is realized in finite-precision arithmetic.

The steepest-edge idea can be stated geometrically. If q enters, the edge direction in full variable space has non-basic component 1 in coordinate q and basic component $-\mathbf{B}^{-1}\mathbf{A}_q$. Its squared Euclidean length is

$$\gamma_q = 1 + \|\mathbf{B}^{-1}\mathbf{A}_q\|_2^2. \quad (7.4)$$

The normalized decrease per unit edge length is therefore $-\bar{c}_q/\sqrt{\gamma_q}$ for a minimization problem. Exact steepest-edge pricing would compute or update all γ_q , while Devex-type rules approximate these weights more cheaply [Mar03, Van20]. Thus, the pricing rule is not part of correctness, but it is central to performance.

8 Variants of the Simplex Method

There is no single simplex algorithm in practice. There is a family of algorithms that share the basis-exchange principle and differ in the invariant they maintain, the structure they exploit, or the representation in which they compute.

The variants in this section should be read as reductions in representation size. Standard form is a universal language, but it can obscure special structure by adding rows, columns, and slack variables. A specialized simplex variant keeps the same optimality certificates while avoiding a representation that would make the linear algebra larger or less sparse than necessary.

8.1 Bounded-Variable Simplex

Many models have explicit bounds $\ell_j \leq x_j \leq u_j$. A bounded-variable simplex method treats variables at lower and upper bounds symmetrically, rather than replacing every upper bound by an additional row. This keeps the basis smaller and preserves sparsity. The same reduced-cost logic applies, but a non-basic variable may enter by moving away from its lower bound or away from its upper bound, and the ratio test must account for both lower and upper blocking events [Mar03, Van20].

One way to formalize this is to write $x_j = \ell_j + z_j$ for variables at their lower bound and $x_j = u_j - z_j$ for variables at their upper bound. The sign of the reduced cost decides which bound is attractive. For a minimization problem, a non-basic variable at its lower bound is eligible to increase

when its reduced cost is negative, while a non-basic variable at its upper bound is eligible to decrease when its reduced cost is positive. The ratio test must then consider both basic variables that hit zero and non-basic bounded variables that hit their opposite bound. This avoids the artificial enlargement that would come from adding a separate slack row for every upper bound [Mar03, Van20].

8.2 Network Simplex

For network-flow problems, bases correspond to spanning-tree structures, and pivots correspond to adding one arc and removing another along a cycle. The network simplex method exploits this combinatorial representation directly, avoiding general-purpose matrix operations when the constraint matrix is a node-arc incidence matrix [AMO93]. This is not a different optimality theory; it is the same complementary slackness and reduced-cost theory written in graph language.

For a minimum-cost flow problem, the dual variables are node potentials. The reduced cost of an arc $a = (u, v)$ has the form

$$\bar{c}_a = c_a + \pi_u - \pi_v, \quad (8.1)$$

up to the sign convention used for the incidence matrix. A spanning-tree basis determines potentials by making tree-arc reduced costs vanish. A non-tree arc with negative reduced cost enters, creating a unique cycle; the leaving arc is selected by the minimum residual capacity along that cycle. Thus, the algebraic basis exchange of simplex becomes the graph operation of adding one arc and deleting one arc [AMO93].

8.3 Dual Simplex in Branch-and-Bound

Dual simplex is the dominant reoptimization tool in mixed-integer programming. When branch-and-bound adds a bound constraint, the parent node's optimal basis often remains dual feasible but becomes primal infeasible. Dual simplex can restore feasibility from that inherited basis much faster than resolving the child node from scratch. This is one reason IP solvers depend heavily on dual simplex even when the user thinks of simplex as a primal vertex-walk algorithm [Mar03, Van20].

The same mechanism appears after cutting planes. Adding a valid inequality to a relaxation typically appends a row and a slack variable. The old primal solution may violate the new inequality, but the old dual multipliers and reduced costs remain a strong starting certificate after the row is incorporated. Dual simplex then repairs the newly infeasible row. This is why simplex is tightly coupled to branch-and-cut implementations, even though the mathematical model being solved changes at every node [Sch86, Mar03, Van20].

8.4 Column Generation and Decomposition

Column generation uses simplex when the full variable set is too large to enumerate. The restricted master problem is solved over a subset of columns, and the reduced-cost calculation becomes a pricing problem that searches for a missing column with negative reduced cost. Dantzig-Wolfe decomposition formalizes this principle for block-structured linear programs [DW60], and Gilmore-Gomory cutting-stock models are the canonical early example [GG61]. The simplex multiplier $\mathbf{y}$ is the interface between the master problem and the pricing subproblem.

Formally, if the full problem has columns indexed by a large set $\mathcal{J}$, the restricted master problem uses only $R \subseteq \mathcal{J}$. After solving the restricted problem, its dual multiplier $\mathbf{y}$ certifies optimality for

the full problem exactly when

$$c_j - \mathbf{A}_j^\top \mathbf{y} \geq 0 \quad \text{for every } j \in \mathcal{J}. \quad (8.2)$$

When $\mathcal{J}$ is too large to scan, Equation (8.2) becomes an optimization problem over the column description. A column with negative reduced cost is added to the restricted master problem; if none exists, the restricted optimum is a full optimum. This is the same reduced-cost theorem from Proposition 3.1, now used as a communication protocol between a master LP and a pricing oracle [DW60, GG61, Mar03].

8.5 Criss-Cross and Other Basis-Exchange Methods

The criss-cross method drops the requirement that intermediate bases be primal feasible or dual feasible. It pivots on violations of either type until both disappear, at which point optimality follows. This viewpoint is useful conceptually because it shows that basis exchange need not be organized around a single feasibility invariant [Ter85]. The price is that the path no longer has the simple monotone objective interpretation of primal simplex or the monotone feasibility-repair interpretation of dual simplex.

The criss-cross viewpoint is also useful historically because it separates basis exchange from monotonicity. Primal simplex is monotone in the primal objective under nondegenerate pivots. Dual simplex is monotone in the corresponding dual repair sense. Criss-cross methods allow neither primal nor dual feasibility to be maintained throughout, but they still terminate under suitable pivoting rules by eliminating sign violations in a finite oriented-matroid framework [Ter85]. Thus, feasibility invariants are sufficient for simplex correctness, but they are not logically necessary for every basis-exchange method.

9 Complexity and Smoothed Behavior

The simplex method has a split complexity theory. Worst-case analysis shows that natural pivot rules can be exponentially slow; polynomial-time linear programming is known through other algorithms; and smoothed analysis explains why simplex can nevertheless behave well under small perturbations.

The important point is that there are three different complexity questions. First, one may ask whether a particular pivot rule takes polynomially many pivots. Second, one may ask whether linear programming as a problem has a polynomial-time algorithm. Third, one may ask why the pivot rules used in practice are usually fast on non-adversarial data. These questions have different answers, and confusing them obscures the subject.

9.1 Worst-Case Exponential Behavior

Klee and Minty constructed deformed cubes on which the simplex method with common pivot rules visits exponentially many vertices [KM72]. Later constructions refined this picture and showed that the phenomenon is not an artifact of one pivot rule alone [Zad09]. These examples should be read as lower bounds on pivoting rules, not as a refutation of simplex as a computational method. They show that local improvement can be forced to follow a long path even when the global optimum is geometrically nearby.

The geometric mechanism is that the objective orientation of the deformed cube makes locally improving edges point along a Hamiltonian path of the cube graph. The optimum may be a single vertex, but the pivot rule receives only local information, and the construction arranges that the local choice keeps following the long path. This demonstrates that the finite termination theorem for simplex cannot be upgraded to a polynomial bound by counting vertices or by appealing to monotone objective improvement alone [KM72, Chv83, Sch86].

9.2 Polynomial-Time Linear Programming

Khachiyan’s ellipsoid method proved that linear programming is polynomial-time solvable [Kha79]. Karmarkar’s algorithm then introduced an interior-point approach that was both polynomial and computationally competitive in many settings [Kar84]. These algorithms changed the theoretical status of LP, but they did not make simplex obsolete. Rather, they clarified that the question “is LP polynomial-time solvable” differs from the question “which basis-exchange rules are polynomial.”

The ellipsoid method works with separation and encoding length rather than with adjacent vertices, while interior-point methods move through the relative interior of the feasible region and follow a central-path geometry. Neither method invalidates the basis certificate at the heart of simplex. Instead, they show that polynomial-time solvability of LP is a property of the optimization problem, not a consequence of every natural local walk on its graph [Kha79, Kar84, Sch86, BT97].

9.3 Smoothed Analysis

Smoothed analysis interpolates between worst-case and average-case analysis by perturbing an arbitrary input slightly and measuring the expected running time on the perturbed instance. Spielman and Teng proved polynomial smoothed complexity bounds for the simplex method under suitable perturbation models [ST04]. The theorem explains why worst-case lower bounds and practical performance can coexist. Pathological instances may exist exactly, but small numerical perturbations can destroy the combinatorial alignment that forces a long pivot path.

Smoothed analysis should not be read as saying that every implementation is automatically fast. It is a theorem about a mathematical perturbation model, a pivot rule, and a condition measure. Its conceptual contribution is that it gives a rigorous framework in which worst-case exponential examples and practical efficiency can both be true. Worst-case analysis controls adversarially exact inputs; average-case analysis controls a distribution chosen in advance; smoothed analysis asks what remains of the worst case after small random noise is added to the data [ST04, Bor87].

10 What the Simplex Method Teaches

The simplex method is best understood as a certificate-producing algorithm. A primal feasible basis gives a vertex, reduced costs give the local objective geometry, and nonnegative reduced costs give a dual certificate of optimality. Primal simplex preserves primal feasibility; dual simplex preserves dual feasibility; the two-phase and Big- M methods manufacture starting certificates; revised simplex turns the tableau algebra into sparse numerical linear algebra; and structural variants exploit special forms of the constraint matrix.

This perspective also explains why simplex remains close to integer programming and cutting planes. Cutting-plane algorithms repeatedly solve LP relaxations after adding constraints; dual simplex is designed exactly for this reoptimization regime. Branch-and-bound repeatedly changes

bounds; dual simplex again repairs the inherited basis. Column generation repeatedly adds variables; simplex reduced costs are precisely the prices that decide which variables matter. Thus, simplex is more than a method for solving a fixed LP. It is the computational engine behind many algorithms that change the LP while preserving enough basis information to continue efficiently.

Acknowledgements. I thank reviewers and readers of this note for corrections, comments, and suggestions that improve the accuracy of the exposition.

References

- [AMO93] R. K. AHUJA, T. L. MAGNANTI, and J. B. ORLIN, *Network Flows: Theory, Algorithms, and Applications*, Prentice Hall, Englewood Cliffs, 1993. (Cited on pp. 4 and 15).
- [BG69] R. H. BARTELS and G. H. GOLUB, The simplex method of linear programming using lu decomposition, *Communications of the ACM* **12** no. 5 (1969), 266–268. <https://doi.org/10.1145/362946.362970>. (Cited on pp. 4 and 13).
- [Bea54] E. M. L. BEALE, Cycling in the dual simplex algorithm, *Naval Research Logistics Quarterly* **1** no. 4 (1954), 269–275. <https://doi.org/10.1002/nav.3800010406>. (Cited on p. 8).
- [BT97] D. BERTSIMAS and J. N. TSITSIKLIS, *Introduction to Linear Optimization*, Athena Scientific, Belmont, 1997. (Cited on pp. 1, 11, 12, and 17).
- [BJS04] R. E. BIXBY, E. A. JENKINS, and D. F. SHANNO, *Linear Programming and Extensions*, Springer, New York, 2004. (Cited on pp. 10 and 14).
- [Bla77] R. G. BLAND, New finite pivoting rules for the simplex method, *Mathematics of Operations Research* **2** no. 2 (1977), 103–107. <https://doi.org/10.1287/moor.2.2.103>. (Cited on pp. 8 and 9).
- [Bor87] K.-H. BORWARDT, *The Simplex Method: A Probabilistic Analysis*, Springer, Berlin, 1987. <https://doi.org/10.1007/978-3-642-61578-8>. (Cited on pp. 4 and 17).
- [Chv83] V. CHVÁTAL, *Linear Programming*, W. H. Freeman, New York, 1983. (Cited on pp. 1, 4, 8, 9, 11, 12, and 17).
- [Dan51] G. B. DANTZIG, Maximization of a linear function of variables subject to linear inequalities, *Activity Analysis of Production and Allocation* (1951), 339–347. (Cited on pp. 1 and 4).
- [Dan63] G. B. DANTZIG, *Linear Programming and Extensions*, Princeton University Press, Princeton, 1963. (Cited on pp. 1 and 4).
- [DW60] G. B. DANTZIG and P. WOLFE, Decomposition principle for linear programs, *Operations Research* **8** no. 1 (1960), 101–111. <https://doi.org/10.1287/opre.8.1.101>. (Cited on pp. 4, 15, and 16).
- [FT72] J. J. H. FORREST and J. A. TOMLIN, Updated triangular factors of the basis to maintain sparsity in the product form simplex method, *Mathematical Programming* **2** no. 1 (1972), 263–278. <https://doi.org/10.1007/BF01584551>. (Cited on pp. 4 and 13).
- [GG61] P. C. GILMORE and R. E. GOMORY, A linear programming approach to the cutting-stock problem, *Operations Research* **9** no. 6 (1961), 849–859. <https://doi.org/10.1287/opre.9.6.849>. (Cited on pp. 4, 15, and 16).
- [Kar84] N. KARMARKAR, A new polynomial-time algorithm for linear programming, *Combinatorica* **4** no. 4 (1984), 373–395. <https://doi.org/10.1007/BF02579150>. (Cited on pp. 4 and 17).
- [Kha79] L. G. KHACHIYAN, A polynomial algorithm in linear programming, *Doklady Akademii Nauk SSSR* **244** (1979), 1093–1096. (Cited on pp. 4 and 17).
- [KM72] V. KLEE and G. J. MINTY, How good is the simplex algorithm?, *Inequalities III* (1972), 159–175. (Cited on pp. 4, 16, and 17).
- [Mar03] I. MAROS, *Computational Techniques of the Simplex Method*, Springer, Boston, 2003. <https://doi.org/10.1007/b97527>. (Cited on pp. 1, 4, 8, 9, 10, 11, 12, 13, 14, 15, and 16).
- [Sch86] A. SCHRIJVER, *Theory of Linear and Integer Programming*, John Wiley & Sons, Chichester, 1986. (Cited on pp. 1, 4, 15, and 17).

- [ST04] D. A. SPIELMAN and S.-H. TENG, Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time, *Journal of the ACM* **51** no. 3 (2004), 385–463. <https://doi.org/10.1145/990308.990310>. (Cited on pp. 1, 4, and 17).
- [Ter85] T. TERLAKY, A convergent criss-cross method, *Optimization* **16** no. 5 (1985), 683–690. <https://doi.org/10.1080/02331938508843073>. (Cited on p. 16).
- [Van20] R. J. VANDERBEI, *Linear Programming: Foundations and Extensions*, 5 ed., Springer, Cham, 2020. <https://doi.org/10.1007/978-3-030-39415-8>. (Cited on pp. 4, 9, 10, 11, 12, 13, 14, and 15).
- [Zad09] N. ZADEH, What is the worst case behavior of the simplex algorithm?, *Polyhedral Computation* (2009), 131–143. (Cited on p. 16).